A Model for Profile Management Through Inference in Trails

André Wagner and Jorge Luis Victória Barbosa
Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950
São Leopoldo - RS, Brazil

Débora Nice Ferrari Barbosa
Universidade Feevale
Pesquisa em Informática na Educação
Rodovia RS-239, 2755
Novo Hamburgo - RS, Brazil

Abstract

Context-aware and ubiquitous systems have the challenge of implicitly collect relevant information about the users, and use this information to understand and predict their behaviour. This will allow the applications to adapt themselves to the user, thus avoiding to overflow the user with inquires and information. This article presents a model that allows applications to register users' actions and infer profile information from these actions, using semantic interoperability and thus allowing different applications of different domains to be able to share information and infer a unified profile.

1. Introdução

Atualmente, a maioria dos usuários interage com uma variedade de sistemas computacionais. Cada vez mais, existe a tendência do aumento da interconectividade entre estes sistemas, devido ao fato da internet estar cada vez mais presente e disponível em qualquer lugar. Assim, pesquisas recentes preveem a integração completa destes sistemas em uma federação de sistemas computacionais, que podem ser descritos como Ambientes Inteligentes [13].

Uma visão para o conceito de ambientes inteligentes foi discutida por Weiser, onde ele apresentou um cenário em que o computador se tornaria invisível e indistinguível de seu ambiente [28]. Neste cenário, a interação entre humanos e computadores ocorreria não como a usual – onde o humano dedica 100% de sua atenção ao computador –, mas constituíra-se de pequenas interações rápidas, geralmente em locais e dispositivos distintos, muitas vezes utilizando a "periferia" da atenção do usuário [29] ou sem ele se dar conta de estar interagindo com um sistema computacional. Segundo o autor, "as tecnologias mais profundas são aquelas que desaparecem. Elas se entrelaçam no tecido do dia-a-dia até que se tornem indistinguíveis dele" [28]. Ele defende que o futuro da computação está em integrar o computador ao mundo real, e chama este novo paradigma de Computação Ubíqua.

No entanto, o aumento dos recursos computacionais disponíveis a qualquer momento e em qualquer lugar oferece o risco da sobrecarga de informações para o usuário. Para resolver este problema, foi criado o conceito de "tecnologia calma" [29]. Neste sentido, a transmissão de informações entre o computador e o usuário devem ocorrer na "periferia" da atenção do usuário, movendo-se para o "centro" da atenção em momentos de urgência. Um exemplo é o motor de um carro. Quando um motorista dirige um carro, sua atenção está focada na estrada, e ele mal percebe o som do motor. No entanto, se houver algum som diferente (o que pode indicar algum problema), a atenção do usuário passa a focar-se no motor.

Para que um sistema seja minimamente intrusivo, é necessário que ele seja capaz de reconhecer a situação do usuário e seus arredores, e adapte seu comportamento de acordo com esta informação. Esta informação é chamada de contexto e sistemas capazes deste tipo de adaptação são chamados sensíveis a contexto [20, 9].

Pesquisas nesta área têm indicado que não apenas é interessante conhecer o contexto do usuário no momento em que um determinado serviço está sendo oferecido, mas que o histórico de contextos visitados pode ser uma informação valiosa na adaptação da aplicação ao usuário. Este histórico de contextos recebe o nome de Trilha [21]. Embora a trilha geralmente esteja vinculada à localização de um usuário, ela possui um conceito mais amplo, que inclui as atividades realizadas, aplicações usadas, conteúdos acessados – entre outros – dentro de um contexto e um período de tempo específico [22].

Para que um aplicativo possa se adaptar a um usuário de forma eficaz, é importante que ele conheça não apenas o histórico de contextos visitados por um usuário, mas possa

compreender quem o usuário é, isto é, quais são suas características. Aplicações deste tipo organizam seu conhecimento a respeito do usuário em Perfis de Usuário. Este conhecimento é armazenado em um formato denominado Modelo de Usuário [24].

Personalização refere-se à disponibilização de produtos, serviços e informações adaptados individualmente. A tecnologia de personalização geralmente envolve programas que aprendam padrões, hábitos e preferências de um usuário. O objetivo principal dos sistemas de personalização modernos é oferecer aos usuários o que elas querem sem requisitar que elas informem isto de forma explícita [17]. Fica claro que este propósito vai ao encontro do foco da computação ubíqua, isto é, o desenvolvimento de uma tecnologia que "desapareça".

Para que um sistema tenha sucesso em obter a intenção do usuário, é necessário que ele conheça o usuário, o contexto que o usuário está inserido e as ações que ele tomou anteriormente quando se encontrava no mesmo contexto ou em contextos semelhantes. Embora existam alguns trabalhos que busquem inferir conhecimento a respeito de um usuário a partir de sua trilha, os trabalhos anteriores operam dentro de um contexto específico (como Educação [4] ou Comércio [12]) ou trabalham com o modelo de usuário de forma estática [16].

Um aspecto relevante para ambientes que operam com múltiplos sistemas independentes é a interoperabilidade. Interoperabilidade é a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizarem as informações que foram trocadas [15]. No âmbito da personalização, interoperabilidade refere-se à capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogêneas e integrar esta informação em um modelo de usuário comum. A interoperabilidade pode ser (i) sintática, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação; ou (ii) semântica, onde as diferenças entre os modelos são resolvidos em nível de conhecimento. [6]

Pesquisas indicam que é desejável que os sistemas do futuro se tornem cada vez mais invisíveis aos seus usuários [29], e que um sistema só pode se tornar invisível se for pró-ativo, e só pode ser pró-ativo se conhecer o usuário [20]. Para que um sistema possa conhecer uma usuário, ele deve conhecer o histórico do usuário, compreender o contexto em que ele está inserido, bem como os dados coletados a respeito dela no mesmo contexto (ou num contexto similar), e ser capaz de processar este conjunto de informações em um perfil de usuário.

Neste contexto está sendo proposto o eProfile, um modelo para a criação automática de um perfis através da inferência em trilhas de entidades. Este modelo possibilita a utilização de um perfil único por diversos sistemas independentes, oferecendo a possibilidade de interoperabilidade semântica entre eles.

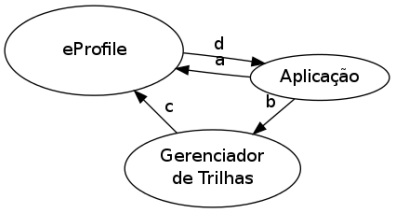

Figura 1: Modelo básico do eProfile

No modelo apresentado, o conceito de Entidade é usado no lugar do conceito clássico de Usuário. Enquanto geralmente o usuário compreende a pessoa que está utilizando o aplicativo, uma entidade refere-se a qualquer objeto (físico ou não) capaz de controlar um aplicativo. Desta forma, uma entidade pode ser uma pessoa, um objeto móvel (como um carro) ou imóvel (como uma sala), ou até mesmo um sistema.

Este artigo está organizado da seguinte forma. Na Seção 2 é proposto o modelo eProfile. A Seção 3 discute os aspectos de implementação. A Seção 4 apresenta um experimento validando o modelo. A Seção 5 descreve trabalhos relacionados, apresentando as contribuições deste modelo. Por fim, a Seção 6 apresenta as considerações finais.

2. Modelo eProfile

A Figura 1 apresenta a visão geral do eProfile, onde cada círculo representa um componente, e as setas indicam o fluxo dos dados.

O primeiro componente representado é a **aplicação**. A aplicação, dentro deste modelo, é qualquer aplicação que está sendo utilizada diretamente por uma entidade. Para que uma aplicação seja considerada apta a este modelo (isto é, possa utilizar as funcionalidades oferecidas pelo eProfile), ela precisa ser capaz de registrar suas informações em um gerenciador de trilhas suportado pelo eProfile, e deve seguir o protocolo de comunicação do eProfile para se comunicar com o mesmo. Seguidos estes pré-requisitos, a aplicação pode ser de qualquer tipo e operar em qualquer tipo de dispositivo que tenha comunicação com a internet.

O segundo componente representado é um **gerenciador de trilhas**. Um gerenciador de trilhas é capaz de registrar eventos realizados por uma entidade, colocando-os em uma ordem cronológica e registando informações relevantes ao evento (como, por exemplo, o contexto na qual o evento foi realizado). O eProfile suporta qualquer gerenciador de trilhas, desde que o protocolo de comunicação com o gerenciador de trilhas esteja codificadado no eProfile.

O terceiro componente da Figura 1 representa o eProfile. As setas indicam o fluxo dos dados. A comunicação tem início quando a aplicação registra-se no eProfile, envi-

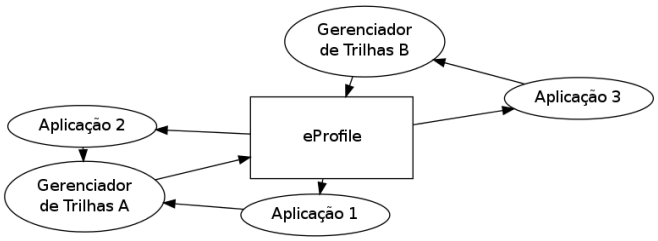

Figura 2: eProfile conectado a mais de um gerenciador de trilhas

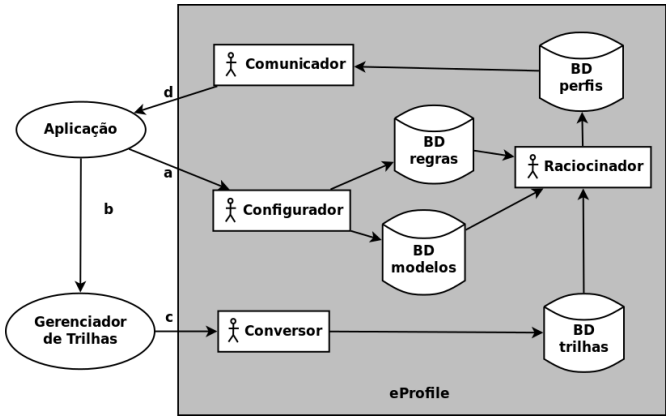

Figura 3: Arquitetura do eProfile

ando (i) o modelo de entidade utilizado pela aplicação e (ii) as regras para construção deste modelo (seta "a"). A partir deste momento, a aplicação passa a registrar os eventos no gerenciador de trilhas (seta "b"). O eProfile periodicamente busca esta informação a partir do gerenciador de trilhas, e constrói ou reconfigura o perfil de entidade (seta "c"). À medida que a aplicação tem necessidade de um perfil atualizado, ela busca este perfil no eProfile (seta "d").

Como o eProfile oferece suporte a várias aplicações, é possível que diferentes aplicações utilizem diferentes gerenciadores de trilhas. Por isso, o eProfile oferece também suporte à comunicação com vários gerenciadores de trilhas, segundo mostrado na Figura 2.

2.1. Arquitetura

A Figura 3 apresenta a arquitetura do eProfile. Os componentes "Aplicação" e "Gerenciador de Trilhas" são os mesmos mostrados na Figura 1, enquanto o serviço "eProfile" foi expandido para mostrar seu funcionamento interno.

Como o eProfile é um modelo que infere informações de forma autônoma a respeito de dados armazenados em uma ontologia, e está organizado em partes independentes, ele utiliza a tecnologia de agentes autônomos.

Cada aplicação tem a possibilidade de montar seu próprio modelo de entidade. Para que o eProfile processe os perfis baseados em um modelo de entidade, ele precisa de três informações: (i) a trilha da entidade, (ii) o modelo de entidade utilizado pela aplicação e (iii) as regras de inferência para o perfil. Isto pode ser observado na Figura 3, onde o agente "Raciocinador" recebe dados destes três bancos. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir o perfil da entidade.

As setas da Figura 3 indicam a direção do fluxo dos dados, onde podem se verificar dois pontos de entrada e um ponto de saída.

O primeiro ponto de entrada são as informações enviadas pela aplicação ao eProfile ou, mais especificamente, ao agente **Configurador**. As informações que devem ser enviadas pela aplicação são as seguintes:

- o modelo de entidade utilizado pela aplicação, ou seja, o formato do perfil da entidade. Uma vez que a trilha da entidade seja processada, as informações inferidas serão armazenadas e disponibilizadas para a aplicação neste formato;
- as regras para inferência do perfil. Estas regras serão executadas em períodos regulares, de modo a atualizar o perfil da entidade. O Configurador tem a função de receber as regras, validá-las e inseri-las na **base de regras**. Juntamente com as regras, é especificado a criticidade da informação. O sistema utiliza esta informação para determinar com que frequência a inferência desta informação será executada.

O segundo ponto de entrada permite o recebimento das trilhas das entidades, que são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 2), onde em cada um deles a trilha tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, para que as trilhas sejam armazenadas de forma padronizada. Este trabalho é realizado por um agente intitulado **Conversor**, que é capaz de realizar a conversão para os formatos de trilha suportados pelo eProfile.

A trilha da entidade é então armazenada em uma ontologia de trilha, chamada **eOntoTrail**. Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser estendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado.

Uma vez de posse das trilhas, do modelo e das regras, o agente **raciocinador** realiza a inferência na trilha, gerando assim um perfil da entidade, que consiste em uma extensão de uma ontologia de perfil básica denominada **eOntoProfile**. Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações

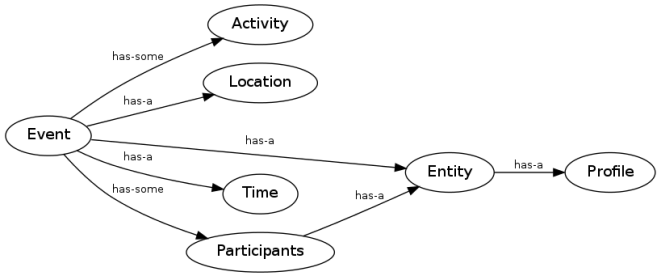

Figura 4: Ontologia de Trilha do eProfile

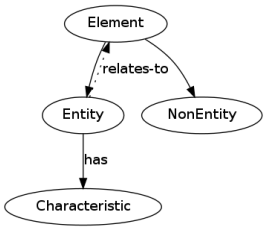

Figura 5: Ontologia de Perfil do eProfile

podem realizar especializações, utilizando-as para armazenar informações próprias. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade, cujos dados são enviados pelo agente **Comunicador**.

2.2. Ontologias

Conforme mencionado na Seção 2.1, o eProfile utiliza duas ontologias: uma ontologia de trilhas e uma ontologia de perfis. Estas são ontologias básicas, estendidas através de subclasses de acordo com o formato da trilha enviada pelo gerenciador (no caso da ontologia de trilha) ou das necessidades da aplicação (no caso da ontologia do perfil).

A ontologia de trilha (eOntoTrail), representada na Figura 4 é alimentada a partir dos diferentes gerenciadores de trilhas suportados, onde o agente Conversor é responsável por receber os dados no formato do gerenciador de trilhas e convertê-los para a eOntoTrail. Assim, o raciocinador tem uma ontologia única sobre a qual deve raciocinar.

Aplicações sensíveis a contexto estão interessadas em saber **quem**, **onde**, **quando** e **o quê** (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo [1]. Assim, uma trilha é representada por um conjunto de eventos caracterizados pelas seguintes classes principais: (i) *Entity*, a identidade da entidade que está realizando a ação, que pode opcionalmente estar vinculada a um perfil; (ii) *Location*, a localização onde está ocorrendo a ação; (iii) *Time*, a data e hora em que ocorreu o evento; (iv) *Participants*, as entidades que estão vinculadas a esta ação; e (v) *Activity:* refere-se à ação que a entidade está executando (ou tentou executar). Esta é a classe mais abrangente da ontologia, e a que possivelmente será a mais especializada pelos gerenciadores de trilha.

A ontologia de perfil (eOntoProfile), representada na Figura 5, é a ontologia utilizada para armazenar os modelos de entidade, bem como as informações de perfil que são inferidas a partir da trilha. Para que a ontologia seja viável dentro do modelo proposto, ela deve ser uma ontologia genérica que permita que suas classes sejam estendidas pelas aplicações. Na figura, um relacionamento entre as classes é mostrado através de uma linha pontilhada, e uma especialização de uma classe é mostrado por uma linha contínua. A eOntoProfile é uma ontologia básica, que é estendida pelas aplicações de modo a montarem seus próprios modelos de entidade.

A classe básica da ontologia é a classe *Element* (elemento). A classe *Element* representa qualquer coisa no mundo real. Ela tem como filhos as classes: (i) *Entity*, que representa uma entidade, isto é, um elemento que tenha um perfil associado a ela; e (ii) *NonEntity*, que representa um elemento que não possui um perfil associado a ela. Por exemplo, uma trilha registra o evento em que uma pessoa entra em uma loja, mas a loja não tem um perfil no eProfile. Neste caso, a loja é uma *NonEntity*. Esta diferenciação é importante porque no primeiro caso, o eProfile tem acesso a uma vasta gama de informações a respeito da entidade, das entidades relacionadas àquela entidade, e assim por diante.

A classe *Characteristic* representa uma característica da entidade, e é a classe que será estendida pelas aplicações para referir-se a qualquer informação relacionada a entidade. Estas características podem ser tanto simples rótulos (como nome no caso de uma pessoa, ou quilometragem no caso de um carro) ou informações mais complexas (como conhecimentos acadêmicos ou últimas rotas viajadas).

2.3. Aplicação de Regras

A aplicação de regras é peça fundamental para o funcionamento do eProfile. Uma regra é uma instrução de inferência registrada por uma aplicação que é executada sobre um conjunto de trilhas de uma entidade, resultando num perfil de entidade de acordo com um modelo de entidade previamente cadastrado pela aplicação.

Para que uma regra possa ser aplicada, a aplicação precisa ter previamente registrado no eProfile: (i) a regra propriamente dita; (ii) em um modelo de entidade, que serve como base para o formato de perfil da entidade; e (iii) a trilha da entidade, sobre a qual a inferência é executada. A partir destas três informações, o raciocinador tem condições de gerar um perfil de entidade, de acordo com a Figura 6.

As regras são sempre definidas pela aplicação, e uma regra é composta pelo seguinte conjunto de informações: (i) nome, (ii) campo do perfil que será atualizado, (iii) a regra, escrita na linguagem OWL [26]; (iv) forma de atualização

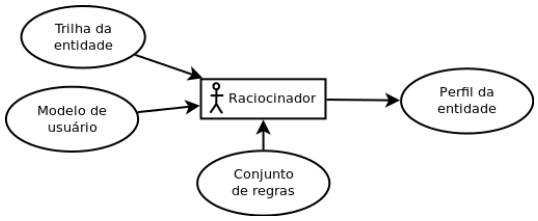

Figura 6: Fluxo de dados para inferência do perfil de entidade

(o valor pode ser sobrescrito ou atualizado de acordo com alguma fórmula); (v) frequência de atualização; e (vi) validade da regra.

3. Aspectos de Implementação

O protótipo do eProfile foi desenvolvido em Python [19]. O Python é uma linguagem genérica de código aberto, orientada a objetos e de alto nível, que oferece suporte a múltiplas arquiteturas. As bases de dados são implementadas utilizando o banco de dados SQLite3 [14]. O SQLite3 é um banco de dados SQL auto-contido transacional, disponível em domínio público. O SQLite3 foi escolhido por apresentar bom desempenho e não necessitar de um servidor separado para sua implantação.

A Figura 6 apresenta o fluxo de dados para a inferência do perfil de uma entidade. No centro desta operação está o agente raciocinador, que utiliza um motor de inferência externo para a execução das regras. O raciocinador utilizado é o FaCT++, um raciocinador de lógica proposicional criado como uma plataforma para utilização de algoritmos de *tableaux* semântico e técnicas de otimização [23]. Este raciocinador foi escolhido por apresentar bom desempenho e suportar todas as funções utilizadas pelo eProfile.

A linguagem de representação de conhecimento utilizada para o armazenamento e processamento das ontologias é o OWL [26], escolhido por ser a linguagem de ontologia mais expressiva definida ou em definição para a Web Semântica [8].

4. Aspectos de Avaliação

Somente é possível avaliar as funcionalidades de uma proposta de infraestrutura construindo aplicações que a utilizem e então avaliá-las, de modo a obter uma avaliação indireta do *framework* [11]. A comunidade científica vem utilizando cenários para validar ambientes sensíveis a contexto (como a abordagem realizada por Dey [10]) e ubíquos (de acordo com Satyanarayanan [20]). Este trabalho segua a mesma abordagem, onde um experimento foi criado para avaliação do eProfile.

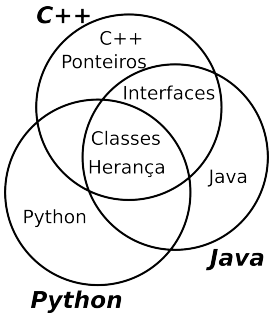

Figura 7: Módulos de estudo obrigatórios

O experimento apresentado é baseado na geração simulada de trilhas de estudantes dentro de um contexto de Educação Ubíqua. Educação Ubíqua pode ser compreendido como processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz [2].

O cenário simula um curso de programação em linguagens orientadas a objeto. O curso consiste em um conjunto de módulos relacionados a orientação a objetos. Destes módulos fazem parte conceitos básicos da matéria (classes, herança, *interfaces* e ponteiros) e linguagens de programação orientadas a objeto (C++, Python e Java). O estudante não é obrigado a concluir todos os módulos. No final do curso, no entanto, ele deve escolher a linguagem de programação que ele utilizará para responder as questões da prova final. Dependendo da linguagem que escolher, ele deve ter concluído, no mínimo, os módulos referentes àquela linguagem.

Por exemplo, caso o aluno decida realizar a prova em Java, ele deve ter estudado classes, herança e *interfaces*, além do Java propriamente dito. Como a linguagem Java não contém o conceito de ponteiros, ele não é obrigado a ter cursado este módulo. As obrigatoriedades dos módulos são representados pela Figura 7. No exemplo proposto, para que o estudante possa realizar a prova na linguagem Python, ele deve ter concluído no mínimo os módulos Classes, Herança e Python.

Neste experimento, os estudantes utilizam um aplicativo de educação ubíqua, usado para a distribuição contextualizada de objetos de aprendizagem. Os conteúdos dos módulos são distribuídos pelo aplicativo como objetos de aprendizagem, e o aplicativo registra em um gerenciador de trilhas cada vez que um módulo é iniciado ou concluído. Tanto o aplicativo quanto o gerenciador de trilhas são simulados.

Cada evento (início ou conclusão de um objeto de aprendizagem) é registrado contendo as seguintes informações: (i) entidade que realizou o evento, (ii) objeto de aprendizagem, (iii) tipo de evento (início ou conclusão), (iii) data e

Entidade	Objeto de aprendizagem	Tipo de evento	Data e Hora	Primeira vez?
Andre	Interfaces	Início	09/07/2012 08:12	Sim
Andre	Classes	Término	09/07/2012 09:28	Sim
Maria	Herança	Término	09/07/2012 09:32	Sim
Andre	Java	Término	09/07/2012 11:14	Sim
Maria	Interfaces	Término	09/07/2012 11:28	Sim
Andre	Ponteiros	Início	09/07/2012 13:58	Sim
Andre	Ponteiros	Término	09/07/2012 14:12	Sim
Andre	Java	Término	09/07/2012 16:18	Não

Tabela 1: Trilha gerada pelo aplicativo

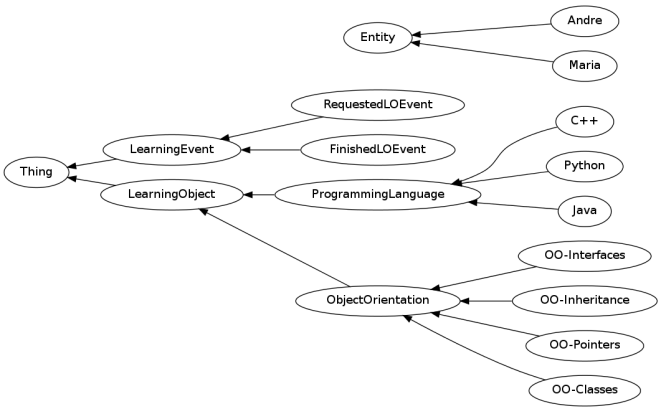

Figura 8: Ontologia do cenário apresentado

hora e (iv) se é a primeira vez que o estudante realiza este tipo de evento.

No cenário simulado, os eventos descritos na Tabela 1 foram registrados pelo aplicativo no gerenciador de trilhas dentro de um determinado período de tempo.

No cenário apresentado, deseja-se atualizar o perfil de uma entidade, inserindo a informação de proficiência dela em determinada disciplina. Esta informação será utilizada posteriormente pela aplicação para determinar se o estudante está apto a realizar o teste.

Para que o eProfile possa ser capaz de inferir estas informações, a aplicação precisa, ao montar as trilhas, enviar informações suficientes para que o eProfile possa montar a ontologia que represente a situação atual. Neste experimento, um exemplo de informação deste tipo são as dependências entre as disciplinas. A Figura 8 mostra a ontologia montada pelo eProfile baseado nas trilhas.

Além das trilhas, a aplicação deve enviar as regras de inferência ao eProfile. Estas regras serão utilizadas pelo eProfile para inferir as trilhas e gerar o perfil. Neste experimento, para que o eProfile possa identificar se uma entidade é proficiente em alguma disciplina, são necessárias três regras: (i) a identificação dos objetos de aprendizagem requeridos

```
1 Class: <el:JavaRequirement>
2     EquivalentTo:
3         <el:Java>
4          or (<el:relatesTo> some <el:Java>)
```

Figura 9: Regra de requerimentos para disciplina

```
1 Class: <el:JavaLessonsTaken>
2     EquivalentTo:
3         <el:FinishedLOEvent>
4          and (<el:hasEntity> some <el:Andre>)
5          and (<el:hasLearningObject> some
6                                <el:JavaRequirement>)
7          and (<el:isFirstTime> value true)
```

Figura 10: Regra de objetos de aprendizagem concluídos

para a proficiência na disciplina; (ii) a identificação dos objetos de aprendizagem concluídos pelo estudante e; (iii) a verificação se os objetos requeridos foram concluídos.

A primeira regra está apresentada na Figura 9, sendo composta por uma classe escrita em OWL[1]. Para que um estudante seja considerado proficiente em Java, ele deve ter concluído os objetos de aprendizagens que fazem relação com a linguagem Java (neste caso classes, herança e *interfaces*, na linha 4), bem como o objeto de aprendizagem da linguagem Java (linha 3).

O segundo passo é a identificação dos objetos de aprendizagem concluídos pelo estudante relacionados ao Java. Esta regra é apresentada na Figura 10. Assim, são considerados apenas os eventos de conclusão de objeto de aprendizagem (linha 3) realizados pela entidade (linha 4), relacionados ao Java (linha 5) e que tenham sido realizados pela primeira vez (linha 7), uma vez que o aluno pode estudar o mesmo material várias vezes. É importante notar que para verificar se a disciplina está relacionada ao Java, foi feita uma relação nas linhas 5 e 6 com a regra apresentada na Figura 9.

Uma vez que estas duas regras estejam criadas na ontologia, o eProfile executa o raciocinador, que gera uma nova ontologia OWL, com as classes inferidas identificadas como filhas das classes criadas.

A terceira regra é uma regra de atualização de perfil é criada (Figura 11, onde são comparados o número de módulos Java (regra da Figura 9) com o número de objetos de aprendizagem relacionados ao Java que foram concluídos pela entidade (regra da Figura 10). Se o número for igual, o usuário é proficiente, caso contrário não, e esta regra é atualizada no perfil da entidade.

[1]Embora o eProfile utilize internamente a sintaxe OWL/RDF [26], a sintaxe Manchester-OWL está sendo usada neste artigo por ser mais compacta e amigável [25].

```
JavaProficience <=
  (Count(SubClasses(JavaRequirement))) equals
          (Count(SubClasses(JavaLessonsTaken)))
```

Figura 11: Regra de proficiência

No experimento realizado, verificou-se que o usuário André não foi considerado proficiente em Java. Isto ocorre pois, de acordo com a Tabela 1, o usuário apenas iniciou mas não concluiu o objeto de aprendizagem sobre *interfaces*, que é um dos pré-requisitos para proficiência em Java.

5. Trabalhos Relacionados

Não foi encontrado nenhum trabalho que faça uso de inferência em trilhas com o objetivo de personalizar a experiência do usuário dentro de um domínio genérico. Assim, o modelo apresentado neste trabalho busca preencher esta lacuna.

O PeLeP [16] suporta aperfeiçoamento automático do perfil de um aprendiz em ambientes de educação ubíqua, utilizando o histórico dos aprendizes para a atualização dos perfis através de inferências. O eProfile utiliza uma estratégia semelhante. No entanto, as regras de inferência são definidas pela aplicação, e não pelo modelo. Esta alteração é necessária para que a inferência passe de um domínio específico (educação) para um domínio genérico. Além disso, no eProfile as regras são dinâmicas (uma aplicação pode definir ou remover regras dinamicamente), ao invés de estarem pré-definidas no modelo. Isto permite a evolução e refino das regras de inferência.

O modelo apresentado por Chellouche et al [7], assim como o eProfile, apresenta um perfil unificado que pode ser utilizado por diversas aplicações. No entanto, ao contrário do eProfile, o modelo assume a interoperabilidade sintática, o que significa que todas as aplicações precisam registrar os eventos utilizando o mesmo formato, o que pode fazer com que a aplicação não consiga registrar o evento com a riqueza de detalhes necessária. Além disso, assim como o PeLeP, o modelo de Chellouche et al [7] está limitado a um domínio específico, que são os serviços multimídia.

O modelo UUCM [18], ao contrário dos outros modelos, trabalha em um domínio genérico. Ele faz isso através da criação de uma ontologia básica que pode ser estendida pelas aplicações. Ao contrário das outras propostas (e do eProfile), o UUCM armazena os perfis no cliente, ao invés do servidor. Isto traz vantagens e desvantagens. Entre as vantagens pode-se citar a privacidade e acesso desconectado. No entanto, esta abordagem limita a interoperabilidade entre sistemas, uma vez que cada cliente fica com uma versão do próprio perfil, restringindo a utilização de um perfil unificado.

A Tabela 2 exibe um comparativo entre o eProfile e os trabalhos relacionados apresentados. O eProfile baseou-se em alguns elementos dos trabalhos citados, tais como o aperfeiçoamento automático de perfis, a utilização de perfis unificados e o funcionamento em um domínio genérico. A partir dos estudos realizados, o eProfile traz como contribuição a utilização de trilhas para a extração de perfis, o que facilita a inferência com sensibilidade a contexto. Além disso, o eProfile permite que cada aplicação registre sua trilha no formato que considerar mais proveitoso para si, permitindo assim que a riqueza de detalhes registrados por uma aplicação possa se traduzir em um perfil mais exato da entidade.

6. Conclusão

O eProfile é um modelo para suporte ao gerenciamento de perfis em ambientes ubíquos que permite a inferência de trilhas de entidades através da definição de regras pelas aplicações. Foi realizado um experimento onde foi simulada a inferência de informações a respeito de estudantes em um ambiente de Educação Ubíqua. As principais contribuições do modelo são: (i) a utilização de trilhas para extração de perfis e (ii) a interoperabilidade semântica, que permite que diferentes aplicações registrem suas trilhas utilizando uma ontologia própria e permitindo com que as aplicações trabalhem em um domínio genérico, isto é, sem um contexto preestabelecido.

O eProfile é uma proposta inicial e requer aperfeiçoamento, especialmente na expansão do protótipo e dos cenários. Como nem todas as funcionalidades estão implementadas no protótipo, alguns procedimentos foram realizados de forma manual na execução do experimento. Além disso, o cenário apresentado mostra o eProfile trabalhando dentro de um domínio específico e, desta forma, não demonstra como múltiplas aplicações trabalhando em contextos diferentes podem gerar um perfil único. Desta forma, novos cenários estão sendo criados de forma que possam validar o modelo de forma total e completa.

Além disso, pretende-se integrar o eProfile a outros sistemas, tais como o sistema de educação ubíqua LOCAL [3] e o gerenciador de trilhas UbiTrail [22].

Referências

[1] G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles. Towards a better understanding of context and context-awareness. In *Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing*, HUC '99, pages 304–307, London, UK, 1999. Springer-Verlag.

[2] D. Barbosa, C. Geyer, and J. Barbosa. Globaledu - an architecture to support learning in a pervasive computing environment. In A. Rettberg and C. Bobda, editors, *New Trends and*

Trabalho	Interope-rabilidade	Perfis dinâmicos	Armaze-namento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM	Sintática	Sim	Cliente	Sim	Não	Genérico
eProfile	Semântica	Sim	Servidor	Sim	Sim	Genérico

Tabela 2: Comparação entre os trabalhos apresentados

Technologies in Computer-Aided Learning for Computer-Aided Design, volume 192 of *IFIP International Federation for Information Processing*, pages 1–10. Springer Boston, 2005.

[3] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa. Local: a model geared towards ubiquitous learning. *SIGCSE Bull.*, 40:432–436, March 2008.

[4] J. L. V. Barbosa, R. M. Hahn, D. N. F. Barbosa, and A. I. da Costa Zanel Saccol. A ubiquitous learning model focused on learner interaction. *International Journal of Learning Technology 2011*, 6(1):62 – 83, 2011.

[5] D. Brickley and L. Miller. FOAF Vocabulary Specification 0.97. Namespace document, January 2010.

[6] F. Carmagnola and F. Cena. User identification for cross-system personalisation. *Inf. Sci.*, 179:16–32, January 2009.

[7] S. A. Chellouche, J. Arnaud, and D. Négru. Flexible user profile management for context-aware ubiquitous environments. In *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, pages 980–984, Piscataway, NJ, USA, 2010. IEEE Press.

[8] M. C. Daconta, L. J. Obrst, and K. T. Smith. *The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management*. Wiley, Indianapolis, May 2003.

[9] A. Dey. Understanding and using context. *Personal and Ubiquitous Computing*, 6(1):4–7, 2001.

[10] A. K. Dey, G. D. Abowd, and D. Salber. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. *Hum.-Comput. Interact.*, 16(2):97–166, Dec. 2001.

[11] W. K. Edwards, V. Bellotti, A. K. Dey, and M. W. Newman. The challenges of user-centered design and evaluation for infrastructure. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, CHI '03, pages 297–304, New York, NY, USA, 2003. ACM.

[12] L. K. Franco, J. H. Rosa, J. L. Barbosa, C. A. Costa, and A. C. Yamin. Mucs: A model for ubiquitous commerce support. *Electronic Commerce Research and Applications*, 10(2):237 – 246, 2011. <ce:title>Special Issue on Electronic Auctions: Strategies and Methods</ce:title>.

[13] D. Heckmann. *Ubiquitous User Modeling*. PhD thesis, Department of Computer Science, Saarland University, Germany, November 2005.

[14] Hwaci. Sqlite home page, 2012. `http://www.sqlite.org/`. Acessado em 05/05/2012.

[15] IEEE. IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries. *IEEE Std 610*, page 1, 1991.

[16] D. Levis, J. L. V. Barbosa, S. C. C. S. Pinto, and D. N. F. Barbosa. Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. *Revista Brasileira de Informática na Educação*, 16(1), 2008.

[17] M. D. Mulvenna, S. S. Anand, and A. G. Büchner. Personalization on the net using web mining: introduction. *Commun. ACM*, 43:122–125, August 2000.

[18] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje. A Multi-Dimensional, Unified User Model for Cross-System Personalization. In *E4PIA Workshop 2004*, 2004.

[19] G. Rossum. Python reference manual. Technical report, Amsterdam, The Netherlands, The Netherlands, 1995.

[20] M. Satyanarayanan. Pervasive computing: Vision and challenges. *IEEE Personal Communications*, 8:10–17, 2001.

[21] J. Silva, J. Rosa, J. Barbosa, D. Barbosa, and L. Palazzo. Content distribution in trail-aware environments. *Journal of the Brazilian Computer Society*, 16:163–176, 2010. 10.1007/s13173-010-0015-1.

[22] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo. Content distribution in trail-aware environments. In *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, pages 15:1–15:8, New York, NY, USA, 2009. ACM.

[23] D. Tsarkov and I. Horrocks. Fact++ description logic reasoner: System description. In *Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006)*, volume 4130 of *Lecture Notes in Artificial Intelligence*, pages 292–297. Springer, 2006.

[24] M. Viviani, N. Bennani, and E. Egyed-Zsigmond. A Survey on User Modeling in Multi-Application Environments. In *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pages 111–116. IEEE, Aug. 2010.

[25] W3C. OWL 2 web ontology language manchester syntax, 2009. `http://www.w3.org/TR/owl2-manchester-syntax/`. Acessado em 08/07/2012.

[26] W3C. OWL web ontology language guide, 2011. `http://www.w3.org/TR/owl-guide/`. Acessado em 05/05/2012.

[27] W3C. RDF vocabulary description language 1.0: Rdf schema, 2011. `http://www.w3.org/TR/rdf-schema/`. Acessado em 05/05/2012.

[28] M. Weiser. The computer for the 21st century. *Scientific America*, pages 94–104, 1991.

[29] M. Weiser and J. S. Brown. Designing calm technology. *PowerGrid Journal*, July 1996.